DEPLOYMENT ARCHITECTURE & END-TO-END OPERATIONS GUIDE V1

Design, deploy, operate and evolve any scalable system
architecture · pipeline · migrations · zero-downtime · rollback · HA/DR · vulnerabilities · checklist

1 · WHAT DEPLOYMENT ARCHITECTURE IS, AND WHY IT IS DESIGNED RATHER THAN ARRANGED

start here — what the term covers, what a deployment actually risks, and how to size the process to the system

The definition

Deployment architecture is **how a system is arranged at runtime and how change reaches it**: which components exist, which boundaries separate them, where state lives, and by what process a code change becomes running behaviour without breaking anything.

Two halves, and both are the subject. The **runtime picture** — tiers, boundaries, data stores. And the **change picture** — pipeline, migrations, strategy, rollback. Most diagrams show only the first, which is why they cannot answer "what happens when we ship on Friday?"

What a deployment actually risks

- **Availability** — dropped connections, restart loops, a full pool during the roll
- **Correctness** — old and new versions running together against one database
- **Data** — a migration that cannot be reversed
- **Security** — a new dependency, a widened permission, an exposed endpoint
- **Cost** — a change in instance size or scaling behaviour
- **Trust** — the second failed release in a week changes how the team behaves

Only the first is usually planned for. The rest are discovered, and typically in that order.

Why frequent deployment is safer, not riskier

The instinct is that shipping less often is safer. The evidence points the other way, and the reason is mechanical rather than cultural.

Infrequent, large releases	Frequent, small releases
Many changes at once, so a failure is hard to attribute	One change, so the cause is obvious
Rollback reverts everything, including what worked	Rollback reverts one thing
The process is rehearsed rarely, so it is unreliable when used	Rehearsed constantly, so it is boring
Long-lived branches, painful merges	Short branches, trivial merges
Fear leads to fewer releases, which makes them larger	Confidence compounds in the other direction

The four measures to hold yourself to. Deployment frequency, lead time for changes, change failure rate, and time to restore service. Throughput and stability improve together in practice — they are not a trade-off, and a process change that moves none of the four was decoration.

Time to restore beats time between failures. A team that reverts in two minutes can take risks a team needing a two-hour restore cannot. Speed of recovery is what permits frequent release.

Tier the system before designing the process

The most expensive mistake in this whole guide is applying a Tier-1 process to a Tier-3 system. Decide the tier first; everything downstream follows.

Tier	Characteristic	Process
1 · Critical	Revenue or safety impact within minutes; regulated	Canary with automated analysis, DR drills, change records, tight RTO/RPO
2 · High	Customer-facing, degradation is visible and costly	Blue/green or canary, automated rollback, SLOs, on-call
3 · Standard	Internal or non-critical; a short outage is tolerable	Rolling, automated tests, basic monitoring
4 · Low	Batch, tooling, experiments	Recreate. Do not over-engineer it

Approval gates and change boards on an internal reporting tool slow everything and teach people to route around the process — which then makes the Tier-1 controls weaker too, because the habit of bypassing is already formed.

How to read the rest of this sheet

Panels 2 and 3 are the change picture. Panels 4 to 8 are how a release is made safe. Panels 9 to 12 are what you decide per system and how you operate it. Panel 13 is the checklist to run against a real deployment.

2 · REFERENCE DEPLOYMENT ARCHITECTURE

drawn as trust and network boundaries, because that is what actually constrains a deployment — not the box names

REQUEST PATH — each dashed boundary is a different blast radius, a different set of credentials, and a different thing that can be deployed independently

CLIENTS — UNTRUSTED
- Web · SPA
- Mobile apps
- API consumers · partners
- Devices · IoT

These are clients, not a server-side tier you deploy

EDGE — PUBLIC
- **DNS** — health-based failover · low TTL
- **CDN · edge cache** — purge on deploy
- **WAF · DDoS · bot** — rules are deployable artefacts
- **Load balancer (L4/L7)** — TLS termination · draining

The traffic-shifting point — every strategy in panel 4 uses

GATEWAY
- **API gateway / ingress** — routing · rate limit · quota · authN · token validation · versioning · transformation — NGINX · Kong · Envoy · cloud LB
- **Identity provider** — OIDC · OAuth 2.1 · SSO — issues the token, validates here

Public subnet — the only tier reachable from internet

APPLICATION TIER — PRIVATE SUBNET, NO PUBLIC IP

Auth	Business svc	Order svc	Notification
stateless	stateless	stateless	stateless

- **Async workers · consumers · schedulers** — deployed separately — drain in-flight work first
- **Stateful services** — ordered rollout, PVCs — not the same deploy

WHAT MAKES THIS TIER DEPLOYABLE WITHOUT DOWNTIME
Stateless — no session or file state on the instance, so any replica can be replaced at any time. Handles SIGTERM: stop accepting, finish in-flight, close connections, exit inside the grace period. Readiness probe gates traffic; liveness does not fail merely because a dependency is down.
A service missing any of these cannot be rolled, whatever strategy the pipeline claims to use.

DATA TIER — ISOLATED SUBNET, REACHABLE ONLY FROM THE APP TIER

Primary DB	Read replicas	Cache	Search
writes · Multi-AZ — schema is the risk	reads · reporting — watch replica lag	Redis · Memcached — cold after restart	OpenSearch — reindex is a migration

- **Object storage** — S3 · blobs · media — lifecycle policies
- **Queue · stream** — Kafka · RabbitMQ · SQS — message schema = contract
- **Backup targets · cross-region replicas** — snapshots · PITR · offsite copy — draw them, or nobody owns them

THE ASYMMETRY THAT GOVERNS EVERY DEPLOYMENT
Everything to the left of this box can be rolled back by redeploying the previous image. Nothing in this box can. A dropped column, a rewritten row, a consumed message and a purged cache are not reversible by a pipeline.
Plan the data change first; the code deployment is the easy half. See panel 5.

EXTERNAL DEPENDENCIES
- Payment gateway
- Email · SMS · push
- Third-party APIs
- Analytics · data platform

Their availability is now yours. Timeout, retry with jitter, circuit-break and degrade — never block a deploy or a request on their worst day.

WHAT A GENERIC ARCHITECTURE DIAGRAM USUALLY LEAVES OUT — and every one of these is a deployment concern

1 · **Network boundaries.** Public, private and isolated subnets, security groups, egress control. Without them "deploy the service" has no defined reachability.
2 · **The load balancer.** Frequently omitted because the gateway is drawn — but the LB is where traffic is actually shifted during blue/green and canary.
3 · **Operator access.** Bastion or zero-trust access, break-glass credentials, audit. How a human reaches production at 3am is part of the architecture.
4 · **Environments.** Dev, staging and production are separate instances of this whole picture. Parity between them is what makes staging predictive.
5 · **Async workers.** Consumers and schedulers deploy on a different rhythm from request-serving services, and must drain in-flight work.
6 · **Backup and DR targets.** If they are not on the diagram, no one owns them, and the restore has never been tested.
7 · **Stateful workloads.** Databases, brokers and anything with a volume follow completely different deployment rules from stateless replicas.
8 · **The clients themselves.** Mobile apps you cannot force-update mean old API versions live in the wild for months. That is a deployment constraint.

Grouping browsers and mobile apps into a server-side "presentation layer" hides point 8 entirely — those are clients you do not deploy and cannot roll back.

PLATFORM & CROSS-CUTTING — deployed and versioned like everything else, and usually the first thing to drift

RUNTIME PLATFORM
- Kubernetes across multiple AZs, or managed compute
- Node pools, autoscaling, PodDisruptionBudgets
- Container registry with retention and scanning

Cluster upgrades are deployments too — and the riskiest ones.

CONFIG & SECRETS
- Config externalised per environment, never baked into images
- Secrets from a managed store, injected at runtime, rotated
- Feature flags as a separate control plane from deployment

Config changes cause outages as often as code changes. Version them.

OBSERVABILITY
- Logs, metrics, traces — correlated by request and release ID
- Dashboards, alerting, SLOs and error budgets
- Deployment markers on every dashboard

If you cannot see the release on the graph, you cannot judge it.

SECURITY CONTROLS
- IAM least privilege; workload identity, no long-lived keys
- Encryption in transit and at rest; mTLS between services
- Admission policy, image signature verification, network policy

Enforce at admission, not only in the pipeline.

RESILIENCE
- Timeouts, retries with jitter, circuit breakers, bulkheads
- Graceful degradation and load shedding
- Health checks that distinguish self from dependency failure

Tested by deliberately breaking things, not by assertion.

COST & CAPACITY
- Requests and limits right-sized; autoscaling on the right signal
- Surge headroom for rolling updates and blue/green duplication
- Cost per environment and per release visible

Missing from most guides — a deploy that doubles spend is a failed deploy.

GOVERNANCE
- Change records, approvals and audit trail where regulated
- Data residency and retention obligations
- Runbooks, on-call rotation, incident process

Design it in — retrofitting an audit trail is far more expensive.

DELIVERY PATH — how a change reaches the picture above. Note that it never writes to production directly.

Source — app code · IaC · config · migrations · policy — one commit, reviewed
▶ **CI** — build · unit · SAST · SCA · secret scan — build once, here only
▶ **Artifact** — image scan · SBOM · sign · push to registry — immutable digest
▶ **Staging** — migrate · deploy · integration · **DAST** — DAST needs it running
▶ **Gate** — policy · approval · change record — automated where possible
▶ **Migrate** — expand phase only · backward compatible · separate from code
▶ **Progressive deploy** — canary → % → full · automated analysis — same artifact as staging
▶ **Verify** — smoke · synthetics · SLO burn · bake time — auto-rollback on breach
▶ **Contract** — stop writing old shape · backfill · then drop — days later, not minutes

THREE PLACEMENT ERRORS THAT MAKE A PIPELINE DIAGRAM WRONG
DAST before deployment — it exercises a running application over HTTP. Before an image exists there is nothing to test.
Scanning before build — image scanning must follow image creation; SCA and SAST come earlier, and they are different tools.
No migration stage — the single most common cause of a deployment that cannot be rolled back.

BUILD ONCE, PROMOTE THE SAME ARTIFACT
The image tested in staging must be the exact digest deployed to production. Rebuilding per environment means the artifact you validated is not the artifact you shipped, and every staging result becomes advisory.
Environment differences belong in configuration, injected at deploy time — never in the build.

HOW TO USE THIS GUIDE
1 · Classify the system — criticality tier, RTO/RPO, compliance scope, traffic profile. Everything downstream follows from that, and a Tier-3 internal tool should not inherit a Tier-1 deployment process.
2 · Size it (panel 10), then choose the architecture above, dropping what you do not need. Most systems do not need a search cluster, a message broker and a service mesh on day one.
3 · Build the pipeline (panels 2, 3) before the system gets large. Retrofitting a delivery process onto a running product is far harder than starting with one.
4 · Decide the data-change process (panel 5) and the rollback plan (panel 7) **before** the first production deploy, not during the first incident.
5 · Then operate: measure deployment frequency, lead time, change failure rate and time to restore. If a change to this process does not move one of those four, it was decoration.

3 · THE DEPLOYMENT PIPELINE, STAGE BY STAGE

what runs where, what it catches, and what it cannot catch at that point

#	Stage	What runs, and why it belongs here
1	**Commit**	Trunk-based, small changes. Pre-commit hooks catch secrets and formatting before CI ever runs. Application code, IaC, config and **migrations travel in the same commit** so they are reviewed together.
2	**Build**	Compile and package once. Stamp the commit SHA and build number into the artifact. Cache dependencies; keep it under a few minutes or people stop watching it.
3	**Unit & component tests**	Fast, isolated, run on every commit. Coverage thresholds that *block*, not just report.
4	**SAST**	Static analysis of source. Catches injection patterns, unsafe deserialisation, hardcoded credentials.
5	**SCA — dependency scan**	Your own code is a minority of what ships. This is usually the highest-yield scan of all, and it is missing from most pipeline diagrams.
6	**Secret scan**	Working tree *and* history. A secret committed and then removed is still leaked.
7	**Build image**	Multi-stage, minimal base, non-root, pinned by digest.
8	**Image scan**	OS and library layers. **Must come after the image exists** — a scan before the build cannot see the base image.
9	**SBOM + sign**	Generate a bill of materials, sign the artifact, attach provenance. This is what later answers "which running services contain this CVE?"
10	**Push to registry**	Immutable tag or digest. Never overwrite an existing tag.
11	**Migrate staging**	Run schema changes as their own step, expand-phase only, before the new code arrives.
12	**Deploy to staging**	Same manifests and process as production. If staging deploys differently, it is not a rehearsal.
13	**Integration & contract tests**	Real dependencies. Contract tests let services deploy independently without an end-to-end suite gating everyone.
14	**DAST**	**Here — against a running application.** Authenticated scanning where it matters. This cannot run earlier in the pipeline.
15	**Performance test**	Against a realistic dataset, with a pass/fail threshold rather than a chart nobody opens.
16	**Gate**	Policy checks automated; human approval only where regulation or risk genuinely requires it. Record the change.
17	**Migrate production**	Expand phase, backward compatible, timed against production-sized data beforehand.
18	**Progressive deploy**	Canary or blue/green, with automated metric analysis deciding promotion. Same artifact digest as staging.
19	**Verify**	Smoke tests, synthetic transactions, SLO burn-rate check, and a bake period before full rollout.
20	**Contract phase**	Days later: stop writing the old shape, backfill, then drop it. The last step of the migration, deliberately separated.

A gate that never blocks is a report. If scans, coverage thresholds and performance tests only produce warnings, they cost pipeline time and change no decisions. Decide which findings stop a release, and let the rest be advisory — explicitly, rather than by everyone learning to ignore the output.

Keep the whole thing fast. Commit to production-ready in well under an hour keeps people watching their changes. Once a pipeline runs for half a day, developers batch work to avoid it, batch size grows, and change failure rate follows — the pipeline's own slowness becomes the delivery risk.

Rollback is not a stage — it is a property of every stage after 17. If any step between 17 and 20 cannot be undone by redeploying the previous digest, that step needed a compensating plan written before it ran.

4 · ENVIRONMENTS & PROMOTION

parity is the point

Environment	Purpose, and what must match production
Local / dev	Fast iteration. Containers so runtime matches; data may be synthetic.
Ephemeral / PR	Created per pull request, destroyed on merge. Catches integration problems before they reach a shared environment.
Staging	**Same topology, same deploy process, same manifests** as production. Scaled down is fine; shaped differently is not.
Production	The only environment that matters. Everything else exists to predict it.
DR	Provisioned from the same IaC. If it is built by hand it will not work when needed.

What breaks parity, in order of frequency

- **Data volume** — a query fast on 10,000 rows and fatal on 100 million. The most common staging blind spot.
- **Traffic and concurrency** — connection pools, locks and race conditions only appear under load.
- **Configuration drift** — a flag set in production months ago that nobody replicated back.
- **Third-party sandboxes** that behave differently from the real service.
- **Manual changes** made in one environment and never codified.

What legitimately varies per environment

Varies	Never varies
Endpoints and hostnames	The artifact digest
Credentials and keys	Application code paths
Replica counts and instance sizes	The deployment mechanism
Rate limits and quotas	Migration scripts
Log level and sampling rate	Health check semantics
Feature flag values	The manifests, beyond substituted values

Promote artifacts, not branches. The same signed digest moves from staging to production. Anything rebuilt for production was never tested.

Staging that nobody trusts is worse than no staging. It costs money, slows every release, and provides false confidence. Either invest in parity or shorten the path and rely on progressive delivery in production with fast rollback — both are defensible; a neglected staging environment is not.

5 · DEPLOYMENT STRATEGIES

and what each costs

Strategy	Behaviour · cost · when to choose it
Recreate	Stop all, start all. Downtime equal to startup time. No extra capacity. Fine for batch jobs, internal tools and anything with a maintenance window.
Rolling	Replace replicas gradually. No downtime, no duplicate environment — but **both versions run simultaneously**, so the change must be backward compatible. The sensible default.
Blue / green	Two full environments, switch traffic at the LB. Instant rollback, simplest mental model. **Doubles infrastructure** during the switch, and shared state (database, cache, queues) is still shared.
Canary	Small traffic share first, expand on metrics. Lowest blast radius, best signal. Needs good telemetry and enough traffic for the sample to mean anything.
Shadow	Mirror real traffic to the new version and discard responses. Excellent for validating performance; must not cause side effects — no writes, no third-party calls.
A/B	Route cohorts to compare outcomes. A product technique, not a safety one.
Feature flag	Deploy dark, release by configuration. Decouples the two entirely and gives the fastest possible revert.

Every strategy shares one weakness: the database. Blue/green switches the application but both colours point at the same schema. Canary runs old and new against one dataset. The strategy protects the code path; nothing protects you from an incompatible data change.

Choosing, in practice

If…	Then
Low traffic	**Blue/green** — canary needs volume for the sample to mean anything
High traffic, good telemetry	**Canary** with automated metric analysis
Cannot afford duplicate capacity	**Rolling**, with strict backward compatibility
Change is risky but reversible	**Feature flag** behind any of the above
Performance is the unknown	**Shadow** first, then canary
Stateful workload	Ordered rolling with PVC care — **not** blue/green
Batch or internal tool	**Recreate**. Do not over-engineer it

Choose by blast radius and traffic volume. Low traffic makes canary metrics meaningless — prefer blue/green. High traffic and good telemetry make canary strictly better. Everything else is preference.

6 · DATABASE & DATA MIGRATIONS

the part of a deployment that cannot be undone by redeploying the previous image — and the most common omission in a deployment guide

The constraint that drives everything here. During a rolling deploy, blue/green switch or canary, **old and new application code run at the same time against one database.** Every schema change must therefore work with both versions simultaneously — not just with the new one.

Expand → migrate → contract

```
// renaming user.name → user.full_name, safely

EXPAND   release 1, deploy before the code
  ADD COLUMN full_name  // nullable, no default backfill
  // old code ignores it, new code not yet live

MIGRATE  release 2
  app writes BOTH columns, reads name
  backfill full_name in batches, throttled
  // safe to roll back — both columns populated

SWITCH   release 3
  app reads full_name, still writes both
  // verify for days, not minutes

CONTRACT release 4, days or weeks later
  app stops writing name
  DROP COLUMN name    // only now, and irreversibly
```

Four releases to rename a column feels excessive until the first time a rollback fails because the previous version wrote to a column that no longer exists. Each step is independently reversible; the combined change is not.

Safe and unsafe changes

Change	Verdict
Add nullable column	**Safe** — old code ignores it
Add table	**Safe**
Add index	**Careful** — build concurrently or it locks writes
Add column with default	**Careful** — may rewrite the table on older engines
Widen a type	**Careful** — usually safe, sometimes a full rewrite
Add NOT NULL constraint	**Unsafe** — old code inserts nulls
Rename column or table	**Unsafe** — breaks whichever version is not updated
Drop column or table	**Unsafe & irreversible** — contract phase only
Narrow a type	**Unsafe** — data loss

Operational rules

- **Migrations run as their own pipeline step**, never on application startup — parallel replicas would each try to run them.
- **Versioned, reviewed, forward-only.** Write the compensating migration rather than relying on a down script nobody has tested.
- **Time it against production-sized data first.** A migration that takes four hours needs a different plan from one that takes four seconds.
- **Backfills run online**, batched and throttled, with progress visible and a way to pause.
- **Lock awareness** — know which statements take an exclusive lock and for how long. A brief lock on a busy table is an outage.
- **Take a backup or snapshot immediately before**, and confirm it completed.

The same rules apply beyond the database. Message schemas must be readable by old and new consumers. Search indices need reindexing strategies. Cache key formats need versioning rather than a flush. Every one of these is a migration with the same expand-contract shape.

7 · ZERO-DOWNTIME MECHANICS

what makes a roll actually safe

A deployment strategy only delivers zero downtime if each instance can be removed and replaced without dropping work. That is a property of the **application**, not of the pipeline.

The shutdown sequence

1. Orchestrator sends `SIGTERM` and removes the instance from the load balancer.
2. Readiness probe starts failing, so no new traffic is routed.
3. **Keep serving in-flight requests** — this is the step most often missed.
4. Stop consuming from queues; finish or return the current message.
5. Flush telemetry, close database and pool connections cleanly.
6. Exit within the termination grace period, before `SIGKILL`.

The grace period must exceed your longest request. A 30-second grace with a 60-second report endpoint kills that request mid-flight on every single deployment — and it will be blamed on the network for months.

The startup side

- **Readiness ≠ liveness.** Readiness gates traffic; liveness restarts the container. Liveness must not fail because a downstream dependency is down, or one outage becomes a restart loop.
- **Startup probes** for slow-booting applications, so liveness does not kill them mid-start.
- **Warm up before accepting traffic** — connection pools, JIT, caches. Cold instances at full traffic look like an outage.
- **Surge capacity** — a rolling update needs room for extra replicas; a cluster at capacity cannot roll.
- **PodDisruptionBudgets** so a node drain cannot take the last healthy replica.

Test it deliberately. Deploy under load and watch for dropped connections and 5xx spikes. Almost every service fails this the first time.

8 · ROLLBACK & RECOVERY

decided before, not during

"We can roll back" is a claim about the whole change, not the code. Write down, before deploying, exactly what reverting undoes and what it does not.

Change type	Reversibility
Stateless code	**Full** — redeploy the previous digest
Config change	**Full** — if versioned
Feature flag	**Instant** — the fastest revert available
Additive schema	**Partial** — new column stays, harmlessly
Backfilled data	**Partial** — needs a compensating job
Destructive schema	**None** — restore from backup
Consumed messages	**None** — offsets have advanced
Third-party side effects	**None** — the payment was taken

Making it fast

- **Automate the trigger** — SLO burn rate or error-rate threshold, not a human noticing a graph.
- **Practise it.** Rollback is the least-exercised path in most systems and runs when everyone is already stressed.
- **Keep previous artifacts** available and deployable; do not prune the registry aggressively.
- **Decide the roll-forward threshold** in advance — for some failures a fix forward is faster and safer than reverting.
- **Name the decision-maker** before the deploy. Debating authority during an incident costs more than the rollback.

Time to restore beats time between failures. A team that can revert in two minutes can take deployment risks that a team needing a two-hour restore cannot — which is why speed of recovery, not caution, is what enables frequent releases.

9 · CONFIGURATION & SECRETS

as risky as code

Configuration

- **Externalised** from the artifact — one image, many environments, config injected at deploy time.
- **Versioned and reviewed** in Git, with the same approval path as code.
- **Validated at startup** — fail fast and loudly on a missing or malformed value rather than at first use.
- **No environment branching in code** — `if (env == "prod")` means production runs an untested path.
- **Drift detection** — reconcile actual against declared, and alert when they differ.

Secrets

- From a managed store at runtime, never in images, repositories, logs or environment dumps.
- **Short-lived and rotated**; prefer dynamic credentials over static ones.
- **Workload identity / OIDC federation** from CI to cloud — removes long-lived keys entirely.
- Least privilege per service; a compromised service should not hold the union of every permission.
- Audit access, and have a tested revocation path.

Config changes cause outages at a rate comparable to code changes, and usually reach production faster because they bypass the pipeline. A rate limit typo, a wrong endpoint, a flag flipped for the wrong cohort — all deploy in seconds with no test suite. Treat configuration as a deployment: reviewed, staged, progressive, observable, revertible.

Feature flags need a lifecycle. Every flag is a permanent branch in your code until deleted. Set an expiry when you create it, and clean up on a schedule — stale flags become untested combinations nobody can reason about.

10 · WHAT TO DECIDE PER SYSTEM

not every system needs every control — match the investment to the criticality tier

Concern	Decide	When it matters most
Criticality tier	Tier the system, and let RTO/RPO, approval depth and on-call follow from it	First decision — everything else derives from it
Resource estimation	Users, RPS, data volume, growth; peak vs average; headroom	Always — at design and at every capacity review
Data store choice	Access patterns first, then the engine; indexing, sharding, pooling	Any system with persistent state
High availability	Multi-AZ, stateless services, health checks, autoscaling	Anything with an availability commitment
Disaster recovery	Cross-region strategy, RTO/RPO, and rehearsed drills	Business-critical and regulated systems
Backup & restore	Automated, offsite, encrypted — and **restore-tested**	Anything holding data you cannot recreate
Security	IAM, least privilege, encryption, secrets, admission policy	Always, every environment
Observability	Logs, metrics, traces, SLOs, alerting with runbooks	Always — you cannot operate what you cannot see
CI/CD	Build once, automated gates, progressive deploy, auto-rollback	Always — it is the delivery mechanism itself
Deployment strategy	Per service, by blast radius and traffic volume	Every release
Data change process	Expand-contract discipline, migration tooling, backfill approach	Any system with a schema — which is nearly all
Rollback plan	What is reversible, the trigger, and who decides	Every deployment, written before it starts
Vulnerability management	Scanning coverage, prioritisation signal, remediation SLA	Always — and audited in regulated sectors
Capacity & autoscaling	Scaling signal, limits, surge headroom for deployments	Variable or growing traffic
Cost	Cost per environment and per request; a budget with alerting	Always — and reviewed monthly, not annually
Compliance & residency	Which regimes apply, where data may live, retention and erasure	Regulated sectors and personal data
Multi-tenancy	Isolation model, noisy-neighbour limits, per-tenant data separation	SaaS and shared platforms
Incident response	On-call, severity levels, comms, postmortems	Anything with users who will notice
Change management	Approvals, change records, freeze policy, audit trail	Regulated and enterprise environments
Dependency lifecycle	Version currency, EOL tracking, upgrade cadence	Always — deferred upgrades compound into rewrites

The most expensive mistake is applying a Tier-1 process to a Tier-3 system. Approval gates, DR drills and change boards on an internal reporting tool slow everything and teach people to route around the process. Tier honestly, then apply the controls the tier actually warrants.

The four columns everyone forgets: cost, compliance, dependency currency and incident response. All four are invisible until a budget review, an audit, a CVE in an unsupported runtime, or a 3am page finds them.

11 · RESOURCE ESTIMATION

start with a baseline

Factor	What to derive from it
Users & traffic	DAU/MAU → concurrent users → average RPS → **peak RPS**. Peak is usually several times average; size for peak, not the mean.
Data volume	Records × size × replication × retention, plus index overhead and growth rate.
Transactions	Write rate, read:write ratio, connection count, transaction duration.
Performance	Latency target at **p95 and p99**, not average. Throughput ceiling from load testing.
Integrations	External call volume, their rate limits, and their latency added to yours.
Retention	Log, metric, backup and audit retention — often the largest storage line.
Environments	Multiply by the number of environments. Non-production is routinely half the bill.
Growth	Projected over 12–18 months, with the point at which the design must change.

Add headroom deliberately, and say how much. Enough to absorb a peak, an AZ failure and a rolling deployment's surge simultaneously — those can coincide. An unstated buffer gets optimised away by the first person reviewing costs.

A worked example — illustrative, not a benchmark

```
// checkout service, from users to pod count
registered users            1,000,000
daily active        10%   =   100,000
sessions per user   1.5   =   150,000 / day
requests per session  40  = 6,000,000 / day

average RPS   6,000,000 / 86,400   ≈  70 rps
peak factor   x5 (evening peak)    ≈ 350 rps

// concurrency from Little's Law: L = λ × W
service time W = 200ms
in-flight  L = 350 × 0.2           =  70 concurrent

// sizing
12 concurrent per pod   → 70 / 12  ≈  6 pods
+ 50% headroom (peak, AZ loss, rollout surge)
                                   =  9 pods

// then verify by load testing. The arithmetic
// gets you a starting point, not an answer.
```

Baseline → measure → optimise → right-size, continuously. The first estimate is always wrong; its purpose is to be close enough to start and specific enough to correct. Re-estimate after every significant traffic or feature change.

12 · HA, DR & BACKUP

three different problems

	Protects against	Mechanism
HA	Instance or AZ failure	Redundancy across AZs, load balancing, health checks, autoscaling, stateless services
DR	Region loss, major incident	Cross-region replication, documented plan, rehearsed drills
Backup	Deletion, corruption, ransomware	Automated, offsite, encrypted, immutable, restore-tested

They do not substitute for each other. Replication faithfully copies a `DROP TABLE` to every replica in milliseconds. HA gives you no protection at all from a bad deployment or a bad command — only a backup does.

Illustrative targets by tier

Tier	RTO	RPO
Critical	≤ 1 hour	≤ 15 minutes
High	≤ 4 hours	≤ 1 hour
Medium	≤ 24 hours	≤ 4 hours
Low	≤ 72 hours	≤ 24 hours

These are starting points, not standards. The numbers come from what the business can tolerate, and they must be agreed with whoever owns that tolerance — then verified by an actual restore, because an unmeasured RTO is an aspiration.

Backup rules that survive an incident

- **Three copies, two media, one offsite** — and at least one **immutable** or air-gapped, because ransomware targets backups first.
- **Separate credentials and account** from production, so one compromise cannot delete both.
- **Restore-test on a schedule**, to a clean environment, timed and recorded.
- **Backup the configuration too** — IaC state, secrets metadata, dashboards, alert rules.
- **Know your retention** against the longest plausible detection delay; corruption found after 30 days needs a 30-day-old copy.

DR patterns by cost: backup-and-restore (cheapest, slowest) → pilot light → warm standby → active-active (fastest, most expensive). Pick from the RTO, then confirm you are willing to pay for it.

What a drill has to prove

- The runbook is followable by someone who did not write it
- Credentials and access work **when the primary region is unavailable** — a surprisingly common failure
- DNS and traffic actually shift, within the expected time
- Data loss is within RPO, measured rather than assumed
- Dependencies — identity, secrets, registries — are also available in the DR region
- Someone recorded the elapsed time, and it met RTO

13 · VULNERABILITY & PATCH MANAGEMENT

the process, and why CVSS severity alone is the wrong thing to queue work by

The loop

Stage	What it involves
1 · Inventory	You cannot patch what you do not know you run. Asset and dependency inventory, SBOMs per artifact, and a registry you can query.
2 · Scan	Source (SAST), dependencies (SCA), images, IaC, running workloads. Continuously — a clean scan at build time says nothing about a CVE published next week.
3 · Prioritise	Severity, exploitation evidence, exposure and reachability together. See opposite.
4 · Remediate	Patch, upgrade, or apply a compensating control. Record an accepted risk with an owner and an expiry when you cannot fix it.
5 · Verify	Re-scan and confirm the fix reached every running instance, not just the repository.
6 · Track	Open counts by severity, mean time to remediate, SLA breaches, trend over time.

Remediation targets — a defensible starting point

Class	Internal	Internet-facing
Known exploited confirmed in the wild	Immediate	**Immediate**
Critical (9.0–10.0)	≤ 30 days	≤ 7 days
High (7.0–8.9)	≤ 60 days	≤ 15 days
Medium (4.0–6.9)	≤ 90 days	≤ 30 days
Low (0.1–3.9)	Next cycle	≤ 90 days

Exposure changes the target. The same CVE on an internet-facing gateway and on an internal batch job are not the same risk, and a single SLA column forces you to treat them identically.

Coverage — what gets scanned, and where

Surface	Scanned at
First-party source	Every commit (SAST)
Dependencies	Every commit, and continuously against new advisories
Container images	At build, and re-scanned in the registry on a schedule
Infrastructure as code	Every commit, before apply
Running workloads	Continuously — drift and newly disclosed CVEs
The running application	DAST against staging, authenticated

CVSS alone over-reports and mis-orders the queue. A CVSS base score describes how bad a vulnerability would be *if exploited*. It says nothing about whether anyone is exploiting it, or whether the vulnerable code path is reachable in your deployment. Queue purely by CVSS and teams drown in Criticals that will never be attacked while a Medium under active exploitation waits behind them.

Three signals, used together

Signal	Answers
CVSS	*How severe is the impact?* Use it as a severity floor — never deprioritise a critical-impact vulnerability on the basis of the other two alone.
EPSS	*How likely is exploitation?* A probability that a CVE will be exploited in the wild in the near term. Use it to order work **within** a severity tier.
KEV catalogue	*Is it already being exploited?* CISA's Known Exploited Vulnerabilities list is confirmation rather than prediction. Anything on it goes to the top regardless of score.
Reachability	*Does our code actually call it?* A vulnerable function in an unused dependency path is a very different risk from one on the request path.
Exposure	*Who can reach the system?* Internet-facing, internal, or isolated.

The practical rule. Known-exploited first, always. Then rank within each severity tier by exploitation likelihood, filtered by whether the path is reachable and how exposed the system is. That ordering turns an unmanageable backlog into a short list — and it is defensible to an auditor, which a raw CVSS queue is not.

Metrics that show whether this works

- **Mean time to remediate**, split by severity and by exposure
- **Open findings past SLA** — the number that should trend to zero
- **Scan coverage** — share of running artifacts with a current scan and an SBOM
- **Backlog age distribution**, not just the count. A stable count can hide an ageing tail
- **Accepted risks with expiry dates**, reviewed rather than silently permanent

Making patching sustainable

- **Automate dependency updates** with a bot raising PRs continuously — small, frequent upgrades beat an annual scramble.
- **Rebuild images on a schedule**, not only on code change. Base images accumulate CVEs while your code sits still.
- **Track runtime and framework EOL dates** as planned work. An unsupported runtime is an unpatchable one.
- **Patch in lower environments first**, then promote through the same pipeline as any other change.
- **Emergency path** — a documented, rehearsed route for a zero-day that bypasses the normal cadence but not the pipeline.

14 · DEPLOYMENT CHECKLIST

before · during · after — and the standing controls that make each deployment routine

- ☐ **Criticality tier agreed**, with RTO/RPO and approval depth following from it
- ☐ **Rollback plan written** — what reverts, what does not, who decides
- ☐ **Migration reviewed** and confirmed backward compatible with the running version
- ☐ **Migration timed** against production-sized data
- ☐ **Backup taken and verified** immediately before a destructive change
- ☐ **Same artifact digest** as the one tested in staging
- ☐ **Config and secrets present** for the target environment, validated at startup
- ☐ **Surge capacity available** for the rollout
- ☐ **Dependencies notified** if the change affects a shared contract
- ☐ **Deployment window appropriate** — and enough people awake to respond
- ☐ **Change record raised** where governance requires it
- ☐ **Progressive strategy chosen** to match blast radius and traffic volume
- ☐ **Automated rollback trigger armed** on error rate or SLO burn
- ☐ **Deployment marker** emitted to dashboards and traces
- ☐ **Canary metrics watched** for the agreed bake period, not just at cutover
- ☐ **Smoke tests and synthetics green** against production
- ☐ **Error rate, latency p99 and saturation** compared to pre-deploy baseline
- ☐ **Queue depth and consumer lag** checked — async paths fail quietly
- ☐ **Logs checked for new error signatures**, not only for volume
- ☐ **Contract phase scheduled** for a later release, with an owner
- ☐ **Feature flags recorded** with an owner and an expiry date
- ☐ **CDN or cache invalidated** where assets changed
- ☐ **Runbook updated** if behaviour or failure modes changed
- ☐ **Cost impact reviewed** after scaling or instance-type changes
- ☐ **Capacity plan current** and headroom stated explicitly
- ☐ **HA verified** — failover actually exercised, not assumed
- ☐ **DR plan documented and drilled**, with the date of the last drill recorded
- ☐ **Restore tested end to end**, and the restore time measured
- ☐ **Graceful shutdown verified** under load — no dropped connections
- ☐ **Grace period exceeds** the longest request the service serves
- ☐ **Readiness and liveness distinct**; liveness independent of dependencies
- ☐ **SLOs defined, alerting on burn rate**, every alert with a runbook
- ☐ **On-call rota staffed** and escalation path known
- ☐ **Secrets rotated on schedule**; no long-lived cloud keys in CI
- ☐ **Scans automated** across source, dependencies, images, IaC and runtime
- ☐ **Vulnerabilities prioritised** by exploitation evidence, not CVSS alone
- ☐ **SBOMs generated and retained** per artifact; images signed and verified at admission
- ☐ **Base images rebuilt on a schedule**; EOL dates tracked as planned work
- ☐ **Environment parity reviewed** — data volume, config and topology
- ☐ **DORA metrics tracked** — frequency, lead time, change failure rate, time to restore
- ☐ **Postmortem process** in place, blameless, with actions tracked to closure

The one that summarises the sheet: **before you deploy, say out loud what you will do if this goes wrong — and confirm that plan still works after the migration has run.** If the answer depends on data that the deployment itself destroys, the change is not ready regardless of how green the pipeline is.